

Hybrid Multi-Agent RAG Architecture

System Design Document: Corporate Competitive Tracker

This document outlines the architecture for an enterprise-grade competitive intelligence tracker. It utilizes a multi-agent framework paired with a Hybrid Large Language Model (LLM) and Small Language Model (SLM) pipeline, leveraging Retrieval-Augmented Generation (RAG) for domain-specific accuracy and massive cost reduction.

1. System Architecture Components

Orchestrator Layer: The Supervisor (Generative LLM)

Acts as the cognitive router. Uses a heavy model (Claude 3.5 or GPT-4o) to interpret user requests, map out execution plans, and route tasks to specialized agent nodes.

Data Gathering Layer: The Scout Agent (API Tools)

Autonomously browses the internet via Tavily or SerpAPI to scrape real-time articles, pricing updates, and press releases. Injects fresh, unstructured text into the shared Agent State.

Retrieval & Embedding Layer: The Brain Agent (FinancialBERT SLM)

Instead of relying on expensive OpenAI embeddings, this agent uses a localized FinancialBERT model to vectorize both live scraped data and historical queries. Optimized specifically for financial semantics.

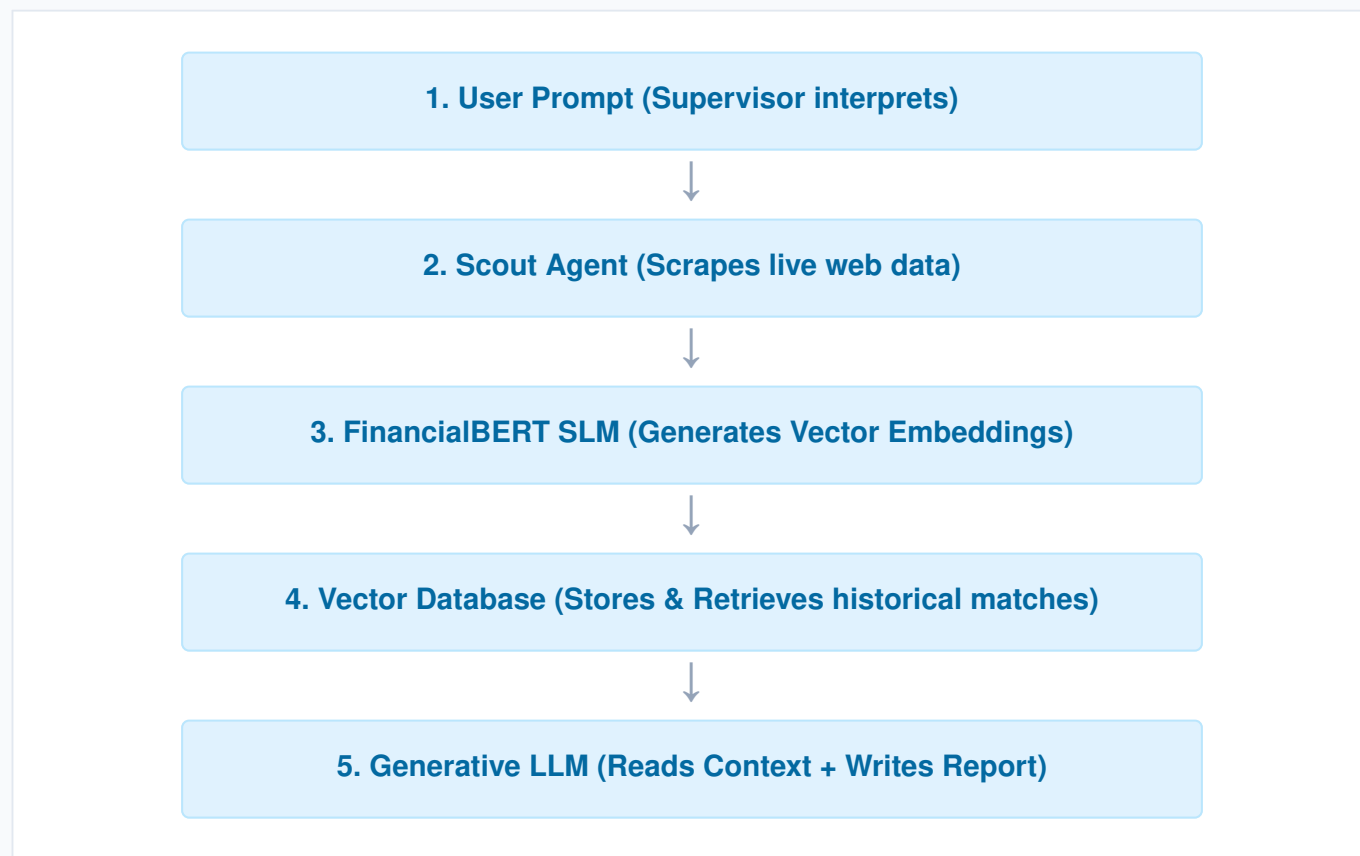
Memory Layer: Vector Database (RAG Storage)

A dedicated database (Qdrant/ChromaDB) that stores FinancialBERT vector embeddings, granting the system long-term memory of competitors' historical actions.

Synthesis Layer: The Writer Agent (Generative LLM)

Reads the finalized state—containing both fresh web data and historical RAG context—and synthesizes it into cohesive, human-readable executive Markdown reports.

2. Data Flow Diagram



3. End-to-End Workflow Example

Scenario: User requests a report on Stripe's new product launches and how they compare to last year's pricing strategy.

1. **Planning:** Supervisor recognizes the need for live news (new products) and historical data (past pricing).
2. **Execution (Live):** Scout searches the web and scrapes a press release published 2 hours ago regarding a new 0.8% enterprise tier.
3. **Vectorization:** The Brain uses FinancialBERT to instantly convert the press release text into math vectors and stores it in the DB.
4. **Retrieval (RAG):** The Brain searches the DB for "Stripe pricing strategy" and retrieves vectors from 6 months ago outlining their flat 2.9% rate.
5. **Synthesis:** The Writer Agent (LLM) reads both the 2-hour old press release and the 6-month old pricing data to write a comparison report.

4. Technology Stack & Value Proposition

Responsibility	Technology Chosen
Agent Orchestration	LangGraph (Stateful, cyclic multi-agent graphs)
Complex Reasoning / Synthesis	Claude 3.5 Sonnet / GPT-4o
Embedding / Context Indexing	FinancialBERT (Local Hugging Face instance)
Web Data Aggregation	Tavily Search API / Beautiful Soup
Long-Term Memory (RAG)	Qdrant or ChromaDB

Why This Enterprise Architecture?

By dividing labor between a massive Generative LLM and a localized FinancialBERT SLM, this architecture achieves a balance rarely seen in beginner AI projects. **FinancialBERT eliminates 90% of embedding API costs** while drastically improving domain-specific contextual matches (e.g., recognizing that "revenue run rate" and "ARR" are semantically identical). Meanwhile, the Generative LLM is preserved strictly for tasks that require deep reasoning and natural language synthesis, maximizing performance while minimizing latency.